UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

[Facultad] — [Carrera]

Curso: [Nombre del curso] — Sección [ ]

Catedrático: [Nombre del catedrático]

Sistema de Recursos Humanos

Proyecto de clase — Fase 1

Especificación Arquitectónica y Diseño

No.	Nombre del integrante	Carné
1	[Nombre completo]	[0000-00-00000]
2	[Nombre completo]	[0000-00-00000]
3	[Nombre completo]	[0000-00-00000]
4	[Nombre completo]	[0000-00-00000]
5	[Nombre completo]	[0000-00-00000]
6	[Nombre completo]	[0000-00-00000]
7	[Nombre completo]	[0000-00-00000]

Guatemala, 27 de septiembre de 2026

Contenido

1. Introducción 4
1.1 Objetivo 4
1.2 Alcance funcional 4
1.3 Tecnologías 4
A. Matriz de Asignación de Responsabilidades (RACI) 6
A.1 Definición del equipo 6
A.2 Matriz RACI por componente 6
A.3 Tareas específicas por integrante 7
B. Justificación de Arquitectura y Patrones 11
B.1 Diagrama de arquitectura de software 11
B.2 Decisión navegacional: SPA frente a MPA 11
B.3 Estrategia de enrutamiento con History API 12
B.4 Distribución del almacenamiento web 13
C. Análisis de Sistemas e Ingeniería de Requisitos 14
C.1 Actores 14
C.2 Diagrama de casos de uso (UML) 14
C.3 Matriz de casos de uso 15
C.4 Especificación de los casos de uso principales 15
D. Diseño de Base de Datos Híbrida (PostgreSQL en Supabase) 20
D.1 Visión general 20
D.2 Diagrama entidad-relación — esquema «empresa» 20
D.3 Diagrama entidad-relación — esquema «reclutamiento» 21
D.4 Diccionario de entidades 21
D.5 Normalización (3FN) 22
D.6 Esquema NoSQL / JSONB 23
D.7 Políticas de seguridad (Row Level Security) 24
E. Especificación de la API REST (Contrato de Interfaz) 26
E.1 Convenciones 26
E.2 Códigos de respuesta 26
E.3 Matriz de endpoints 27
E.4 Ejemplos 28
F. Plan de Seguridad y Mitigaciones OWASP 30
F.1 Manejo de sesiones y autenticación 30

F.2 Matriz de mitigación de vulnerabilidades 30
F.3 Manejo seguro de secretos 31
Anexo. Referencias del proyecto 33

Si el índice aparece vacío, en Word haz clic derecho sobre él y elige «Actualizar campo».

1. Introducción

El **Sistema de Recursos Humanos** es una aplicación web que centraliza la gestión del personal de una empresa guatemalteca y su proceso de reclutamiento. Administra expedientes de empleados, contratos, nómina conforme a la legislación de Guatemala (IGSS, ISR, bonificación incentivo, Aguinaldo y Bono 14), vacaciones y permisos, evaluaciones de desempeño, liquidaciones laborales y constancias en PDF, además de un portal público de empleo donde los postulantes envían su CV.

La solución usa **dos bases de datos lógicas** en PostgreSQL (Supabase): el esquema `empresa`, con la información interna de la organización, y el esquema `reclutamiento`, con los datos de los prospectos. Ambos están protegidos con Row Level Security (RLS) y control de acceso basado en roles (RBAC).

1.1 Objetivo

Diseñar y construir una aplicación web segura, con arquitectura cliente → API REST propia → Supabase, que cumpla los requerimientos técnicos obligatorios del curso: SPA con History API, almacenamiento web (localStorage, sessionStorage y cookies seguras), API REST propia consumida con fetch/async-await, base de datos híbrida (relacional + JSONB) con RLS, y mitigación de vulnerabilidades OWASP.

1.2 Alcance funcional

Módulo	Funcionalidades principales
Autenticación y usuarios	Inicio y cierre de sesión, recuperación de contraseña, invitación de usuarios por correo, 5 roles, eliminación de cuentas
Empleados	Expediente, contratos e historial salarial, documentos digitales, evaluaciones de desempeño
Organización	Departamentos (con jefe), puestos con banda salarial y datos de la empresa
Vacaciones y permisos	Solicitudes del empleado y aprobación del jefe o de RRHH
Nómina	Planilla mensual, Aguinaldo y Bono 14, cierre de período, recibos imprimibles y archivo para el banco
Liquidación	Indemnización, prestaciones proporcionales, vacaciones y salario pendiente al dar de baja
Constancias	Constancia laboral y de ingresos en PDF con código de verificación
Reclutamiento	Vacantes, postulantes, CV, antecedentes, referencias, proceso de selección (Kanban), entrevistas y contratación
Portal de empleo	Página pública de vacantes y postulación con CV en 3 pasos
Administración	Panel con indicadores y gráficas, bitácora de auditoría con búsqueda en JSONB

1.3 Tecnologías

Capa	Tecnología	Uso
Frontend	HTML5, CSS3 puro, JavaScript ES6+ (módulos), Vite	SPA, History API, almacenamiento web, fetch/async-await
Backend	Node.js 22 en Netlify Functions	API REST propia /api/v1, validación, cookies HttpOnly, SQL directo
Base de datos	PostgreSQL 17 (Supabase)	Esquemas empresa y reclutamiento, JSONB, RLS, funciones y triggers
Identidad	Supabase Auth	Usuarios, JWT, invitaciones y recuperación por correo (SMTP)
Archivos	Supabase Storage	Buckets privados para CV y documentos
Despliegue	Netlify + GitHub	Integración y despliegue continuo desde la rama main

A. Matriz de Asignación de Responsabilidades (RACI)

A.1 Definición del equipo

El equipo tiene 7 integrantes. Cada uno es dueño de un área técnica del proyecto y de las tareas específicas detalladas en A.3.

Integrante	Nombre	Carné	Rol asignado
I1	[Nombre completo]	[Carné]	Líder de proyecto · Integración y despliegue
I2	[Nombre completo]	[Carné]	Frontend · Interfaz HTML5 y CSS3
I3	[Nombre completo]	[Carné]	Frontend · SPA, History API y almacenamiento web
I4	[Nombre completo]	[Carné]	Backend · API REST, autenticación y usuarios
I5	[Nombre completo]	[Carné]	Backend · Lógica de negocio, archivos y portal
I6	[Nombre completo]	[Carné]	Base de datos · DER, JSONB y RLS
I7	[Nombre completo]	[Carné]	Seguridad OWASP · Pruebas y QA

A.2 Matriz RACI por componente

R = Responsable de ejecutar · **A** = Aprobador final (uno por componente) · **C** = Consultado para apoyo técnico · **I** = Informado del avance.

Componente del sistema	I1	I2	I3	I4	I5	I6	I7
Planificación, seguimiento y documentación	R/A	C	C	C	C	C	C
Repositorio GitHub y despliegue en Netlify	R/A	I	I	C	I	I	C
Configuración de Supabase (Auth, SMTP, plantillas)	R/A	I	I	C	I	C	C
Maquetación HTML5/CSS3 y diseño responsivo	A	R	C	I	I	I	C
Tema claro/oscuro (localStorage) y panel de gráficas	I	R/A	C	I	C	C	I
SPA y enrutamiento con History API	A	C	R	I	I	I	C
Consumo de la API con fetch/async-await	I	C	R	A	C	I	C
Portal de empleo y borrador en sessionStorage	I	C	R/A	I	C	I	C
API REST — autenticación, cookies HttpOnly y usuarios	I	I	C	R/A	C	C	C
API REST — CRUD genérico y códigos HTTP	I	I	C	R/A	C	C	C

Componente del sistema	I1	I2	I3	I4	I5	I6	I7
Nómina, liquidación y constancias PDF	I	C	I	C	R/A	C	C
API REST — archivos y portal público	I	I	C	A	R	C	C
DER relacional (3FN) y migraciones SQL	I	I	I	C	C	R/A	I
Columnas JSONB y consultas SQL directas	I	I	I	C	C	R/A	C
Políticas RLS y funciones por rol	I	I	I	C	C	R	A
Mitigación XSS, CSRF, inyecciones y CSP	I	C	C	C	C	C	R/A
Manejo de secretos y variables de entorno	C	I	I	C	I	I	R/A
Pruebas automáticas y QA	A	C	C	C	C	C	R

A.3 Tareas específicas por integrante

Cada tarea tiene un identificador (T<integrante>.<número>), el archivo o entregable que produce y el criterio con el que se da por terminada.

I1 — Líder de proyecto · Integración y despliegue

ID	Tarea	Entregable	Criterio de aceptación
T1.1	Elaborar el cronograma de las fases, asignar fechas a cada tarea y dar seguimiento semanal del avance	Cronograma y minutas de reunión	Cada tarea tiene responsable y fecha; el avance se revisa cada semana
T1.2	Crear y administrar el repositorio privado en GitHub: ramas `main` y de trabajo, revisión y fusión de solicitudes de cambios	Repositorio `sistema-rrhh`	Ningún cambio llega a `main` sin revisión ni sin una vista previa probada
T1.3	Configurar el proyecto de Supabase: región, SMTP de Gmail, URL del sitio y plantillas de correo en español	`supabase/templates/*.html`	La invitación y la recuperación llegan en español y abren el propio sitio
T1.4	Configurar Netlify: comando de compilación, redirecciones `/api/v1/*` y SPA, y variables de entorno secretas	`netlify.toml`	Despliegue automático desde `main` y Deploy Preview por cada solicitud de cambios
T1.5	Integrar el trabajo de todos y dirigir la prueba final en la vista previa antes de publicar	Lista de verificación de integración	Todos los flujos de la sección C pasan en el Deploy Preview
T1.6	Consolidar, revisar y entregar el documento de la Fase 1	Este documento en PDF	Entregado el 27 de septiembre de 2026

I2 — Frontend · Interfaz HTML5 y CSS3

ID	Tarea	Entregable	Criterio de aceptación
T2.1	Maquetar la estructura base: `index.html`, menú lateral, encabezado y contenedor de vistas	`index.html`, `src/layout.js`	HTML5 semántico, sin frameworks de CSS
T2.2	Definir el sistema visual con variables CSS: colores, tipografía, tablas, formularios, botones y modales	`src/styles/main.css`, `src/ui.js`	Todas las vistas usan los mismos componentes
T2.3	Implementar el tema claro/oscuro guardado en localStorage y aplicado antes de pintar la página	`src/tema.js`, `public/tema-inicial.js`	Al recargar no hay parpadeo y se conserva la preferencia
T2.4	Adaptar el diseño a celular, tableta y escritorio con media queries	Reglas responsivas en `main.css`	Utilizable a 375 px de ancho sin desplazamiento horizontal
T2.5	Construir las vistas de empleados, expediente, organización y vacaciones/permisos	`src/views/empleados.js`, `empleado.js`, `organizacion.js`, `ausencias.js`	Formularios con validación HTML5 y mensajes de error claros
T2.6	Construir el panel de indicadores con gráficas propias en canvas	`src/views/dashboard.js`, `src/graficas.js`	Muestra los datos de `/reportes/resumen` y se redibuja al cambiar el tamaño

I3 — Frontend · SPA, History API y almacenamiento web

ID	Tarea	Entregable	Criterio de aceptación
T3.1	Programar el enrutador con `pushState`, `replaceState` y el evento `popstate`, e interceptar los enlaces internos	`src/router.js`	Se navega sin recargar; los botones Atrás/Adelante funcionan
T3.2	Proteger las rutas según la sesión y el rol, y redirigir al inicio de sesión cuando la API responde 401	`src/main.js`, `src/auth.js`	Un empleado no puede abrir por URL una vista de administración
T3.3	Escribir la capa cliente de la API con `fetch` y `async/await`: token CSRF en memoria, sanitización previa al envío y manejo de errores	`src/api.js`	Ninguna vista llama a Supabase directamente; los errores se muestran al usuario
T3.4	Construir el portal público de empleo y el formulario de postulación en 3 pasos	`src/views/empleos.js`	Se puede postular con CV en PDF sin iniciar sesión
T3.5	Guardar el borrador del formulario de postulación en sessionStorage	Clave `rrhh:postulacion:{vacante}`	Al recargar se recupera el borrador; se borra al enviar
T3.6	Manejar los enlaces de correo (`token_hash`) y la pantalla de restablecer contraseña	`src/main.js`, `src/views/restablecer.js`	La invitación y la recuperación terminan con sesión iniciada

I4 — Backend · API REST, autenticación y usuarios

ID	Tarea	Entregable	Criterio de aceptación
T4.1	Implementar el enrutador de la API `/api/v1` y su despliegue como Netlify Function (Node 22)	`server/app.js`, `netlify/functions/api.mjs`	Rutas desconocidas responden 404 en JSON
T4.2	Implementar `/auth`: login, sesión, logout, recuperar, verificar y cambio de contraseña	`server/rutas/auth.js`	Respuestas 200/400/401 según el contrato de la sección E
T4.3	Emitir las cookies de sesión `HttpOnly; Secure; SameSite=Strict` y renovar el token automáticamente	`server/http.js`, `server/sesion.js`	`document.cookie` queda vacío en el navegador
T4.4	Implementar `/usuarios`: invitar por correo con rol y eliminar cuentas, registrándolo en la bitácora	`server/rutas/usuarios.js`	Solo el rol admin puede usarlo (403 para los demás)
T4.5	Implementar el CRUD genérico con lista blanca de 23 recursos, filtros, orden y paginación	`server/recursos.js`	Un recurso o método no permitido responde 404 o 405
T4.6	Traducir los errores de PostgreSQL a códigos HTTP (400, 403, 404, 409)	Función `errorDeBaseDatos`	Nunca se muestra al cliente un error interno de la base de datos

I5 — Backend · Lógica de negocio, archivos y portal

ID	Tarea	Entregable	Criterio de aceptación
T5.1	Programar el cálculo de nómina de Guatemala: IGSS 4.83 % / 10.67 %, IRTRA, INTECAP, bonificación Q250 e ISR	`src/lib/nomina-gt.js`	Coincide con los casos de prueba de `tests/nomina-gt.test.js`
T5.2	Programar Aguinaldo, Bono 14 y la liquidación (indemnización con 1/6, vacaciones y salario pendiente)	`src/liquidacion.js`, `src/views/periodo.js`	La liquidación queda guardada en JSONB con su desglose
T5.3	Generar las constancias laboral y de ingresos en PDF con código de verificación y monto en letras	`src/constancias.js`, `src/lib/letras.js`	El PDF se descarga con el código registrado en la base de datos
T5.4	Implementar `/archivos`: subida multipart con verificación de firma del archivo, URL firmada y borrado	`server/rutas/archivos.js`	Rechaza archivos de más de 5 MB o con un tipo falso
T5.5	Implementar `/publico`: vacantes, datos de la empresa y postulaciones con honeypot y límite por IP	`server/rutas/publico.js`	Más de 5 envíos por hora desde la misma IP responden 429
T5.6	Implementar la contratación de un postulante (`POST /postulaciones/{id}/contratar`)	`server/app.js`	Crea el empleado enlazado al postulante y responde 201

I6 — Base de datos · DER, JSONB y RLS

ID	Tarea	Entregable	Criterio de aceptación
T6.1	Diseñar el DER de los esquemas `empresa` y `reclutamiento` en tercera forma normal	Figuras 3 y 4 (sección D)	Cada tabla tiene PK, FK y restricciones de unicidad
T6.2	Escribir las migraciones SQL versionadas del esquema inicial, liquidaciones, constancias y portal	`supabase/migrations/*.sql`	Se aplican en orden sobre una base vacía sin errores
T6.3	Diseñar las columnas JSONB de la bitácora (antes/después) y de la liquidación (ingresos/descuentos)	Triggers de auditoría	Cada INSERT, UPDATE y DELETE queda registrado automáticamente
T6.4	Crear las funciones auxiliares de seguridad: `tiene_rol`, `empleado_actual`, `es_de_mi_equipo` y `vacante_de_mi_depto`	Funciones SQL `security definer`	Sin permiso EXECUTE para usuarios anónimos
T6.5	Escribir las 57 políticas RLS (55 en tablas y 2 en Storage) por rol (admin, rrhh, nomina, jefe, empleado)	Políticas de la sección D	Cada rol ve y modifica solo lo que le corresponde
T6.6	Escribir las consultas SQL directas del backend: resumen del panel y búsqueda en la bitácora JSONB	`server/db.js`, `server/rutas/reportes.js`	Se ejecutan con el rol `authenticated`, por lo que RLS se sigue aplicando

I7 — Seguridad OWASP · Pruebas y QA

ID	Tarea	Entregable	Criterio de aceptación
T7.1	Implementar la sanitización y validación de entradas en el servidor	`server/sanear.js`	Etiquetas HTML y caracteres de control se eliminan antes de guardar
T7.2	Implementar la protección CSRF: token `X-CSRF-Token` firmado con HMAC, cabecera `X-Requested-With` y verificación del `Origin`	`server/sesion.js`, `server/http.js`	Una petición sin token válido o desde otro sitio responde 403
T7.3	Configurar las cabeceras de seguridad y la política CSP	Sección `[[headers]]` de `netlify.toml`	No se ejecutan scripts externos ni la página puede incrustarse en otro sitio
T7.4	Definir el manejo de secretos: variables de entorno, `.env` fuera del repositorio y ninguna clave en el navegador	`.env.example`, `.gitignore`	El bundle publicado no contiene claves de Supabase
T7.5	Escribir y mantener las pruebas automáticas de la API, la nómina y los montos en letras	`tests/*.test.js` (24 casos)	`npm test` pasa antes de cada fusión a `main`
T7.6	Ejecutar la prueba manual por rol y revisar los asesores de seguridad de Supabase	Informe de QA	Sin advertencias de seguridad abiertas antes de publicar

B. Justificación de Arquitectura y Patrones

B.1 Diagrama de arquitectura de software

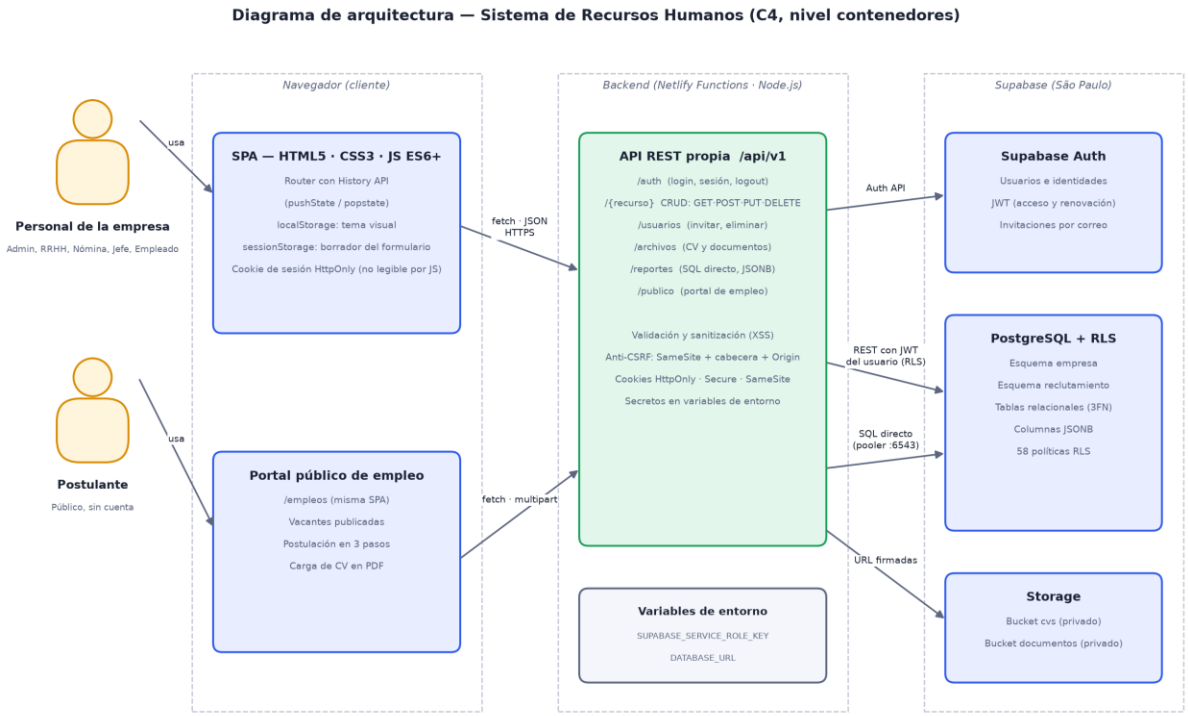


Figura 1. Diagrama de contenedores (modelo C4): Cliente (Web) → API REST (Backend) → Supabase.

La arquitectura sigue tres capas desacopladas:

- **Cliente (navegador):** una SPA en HTML5, CSS3 y JavaScript ES6+. No contiene ninguna clave de Supabase ni se conecta directamente a la base de datos; toda operación pasa por la API propia mediante `fetch` con `async/await`.
- **Backend (API REST propia):** funciones serverless de Netlify en Node.js publicadas en `/api/v1` en el mismo dominio que la SPA. Valida y sanea las entradas, administra la sesión en cookies HttpOnly, aplica protección CSRF y consulta Supabase con el token del usuario (RLS) o mediante SQL directo.
- **Datos (Supabase):** PostgreSQL con dos esquemas lógicos (empresa y reclutamiento), Supabase Auth para identidades y JWT, y Storage para archivos privados. La seguridad se aplica en la base de datos con RLS, de modo que ni un error en el frontend ni en la API puede exponer datos de otro rol.

B.2 Decisión navegacional: SPA frente a MPA

Se evaluaron ambos patrones según los casos de uso del sistema:

Criterio	SPA (elegida)	MPA
Tipo de uso	Aplicación de trabajo diario con muchas pantallas y cambios frecuentes entre ellas	Sitios de contenido con pocas interacciones

Criterio	SPA (elegida)	MPA
Velocidad percibida	Solo se descargan datos (JSON); la interfaz no se recarga	Cada clic recarga la página completa
Estado de la interfaz	Se conservan menús, filtros, formularios y la sesión en memoria	El estado se pierde en cada navegación
Interacciones ricas	Tablero Kanban con arrastrar y soltar, gráficas, modales, formularios por pasos	Más complejas de implementar
Consumo de API	Natural: la SPA consume la API REST propia	Requiere renderizado en el servidor
SEO	No es relevante: el sistema es privado (solo el portal de empleo es público)	Ventaja cuando el SEO importa
Carga inicial	Mitigada con carga diferida por vista (code splitting): 28 KB iniciales	Menor por página

Conclusión: se eligió una **SPA** porque el sistema es una herramienta interna de uso intensivo, con muchas vistas interrelacionadas y componentes interactivos. Las desventajas típicas de una SPA se mitigan con la History API (URLs reales, botones Atrás/Adelante, enlaces directos), carga diferida de cada pantalla y un fallback del servidor que entrega `index.html` en cualquier ruta.

B.3 Estrategia de enrutamiento con History API

- **URLs limpias:** cada pantalla tiene una ruta real (`/empleados`, `/empleados/{id}`, `/nomina/{id}`, `/empleos?vacante={id}`), sin `#`.
- **pushState:** al navegar se agrega una entrada al historial y se renderiza la vista sin recargar la página.
- **replaceState:** en redirecciones (por ejemplo, un usuario sin sesión que abre `/empleados` es enviado a `/login`) se sustituye la entrada actual para no ensuciar el historial.
- **popstate:** los botones Atrás/Adelante del navegador disparan este evento y el router muestra la vista correspondiente.
- **Intercepción de enlaces:** los clics en `<a href="/ruta">` internos se interceptan para navegar con pushState; Ctrl+clic y enlaces externos funcionan normalmente.
- **Control de acceso por rol:** cada ruta declara los roles permitidos; si no corresponde, el router redirige a la página de inicio del rol.
- **Fallback del servidor:** `netlify.toml` reescribe cualquier ruta a `index.html` para que las URLs se puedan recargar o compartir.

Ruta	Vista	Roles
/login	Inicio de sesión	Público
/empleos	Portal de empleo	Público
/	Panel de inicio	admin, rrhh, nomina, jefe
/mi-portal	Portal del empleado	Usuarios con expediente vinculado

Ruta	Vista	Roles
/empleados, /empleados/{id}	Lista y expediente	admin, rrhh, nomina, jefe
/organizacion	Departamentos, puestos, datos de la empresa	admin, rrhh
/ausencias	Vacaciones y permisos	admin, rrhh, nomina, jefe
/nomina, /nomina/{id}	Períodos y recibos	admin, rrhh, nomina
/vacantes, /postulantes, /proceso	Reclutamiento	admin, rrhh, jefe
/usuarios, /auditoria	Administración	admin

B.4 Distribución del almacenamiento web

Mecanismo	Dato que guarda	Duración	Justificación
localStorage	Tema visual (`rrhh:tema`: sistema, claro u oscuro)	Permanente en el navegador	Preferencia de interfaz no sensible que debe sobrevivir entre visitas
sessionStorage	Borrador del formulario de postulación de 3 pasos (`rrhh:postulacion:{vacante}`)	Mientras la pestaña esté abierta	Datos volátiles del asistente: sobreviven a una recarga pero se borran al cerrar la pestaña
Cookie `rrhh_at` (HttpOnly, Secure, SameSite=Strict)	Token de acceso JWT	1 hora (se renueva solo)	JavaScript no puede leerla (mitiga XSS); no viaja desde otros sitios (mitiga CSRF)
Cookie `rrhh_rt` (HttpOnly, Secure, SameSite=Strict)	Token de renovación	Sesión del navegador, o 30 días con «Mantener sesión iniciada»	Permite renovar la sesión sin volver a pedir contraseña
Memoria (JavaScript)	Perfil y rol del usuario	Hasta recargar	Se vuelve a pedir a la API con GET /api/v1/auth/sesion

C. Análisis de Sistemas e Ingeniería de Requisitos

C.1 Actores

Actor	Descripción	Rol en el sistema
Administrador	Responsable del sistema; tiene acceso total	admin
Recursos Humanos	Gestiona personal, organización y reclutamiento	rrhh
Nómina	Calcula, revisa y cierra las planillas	nomina
Jefe de área	Supervisa a su equipo: aprueba ausencias, evalúa y consulta las vacantes de su departamento	jefe
Empleado	Consulta sus recibos, solicita vacaciones y descarga constancias	empleado
Postulante	Persona externa que postula desde el portal público, sin cuenta	— (anónimo)

C.2 Diagrama de casos de uso (UML)

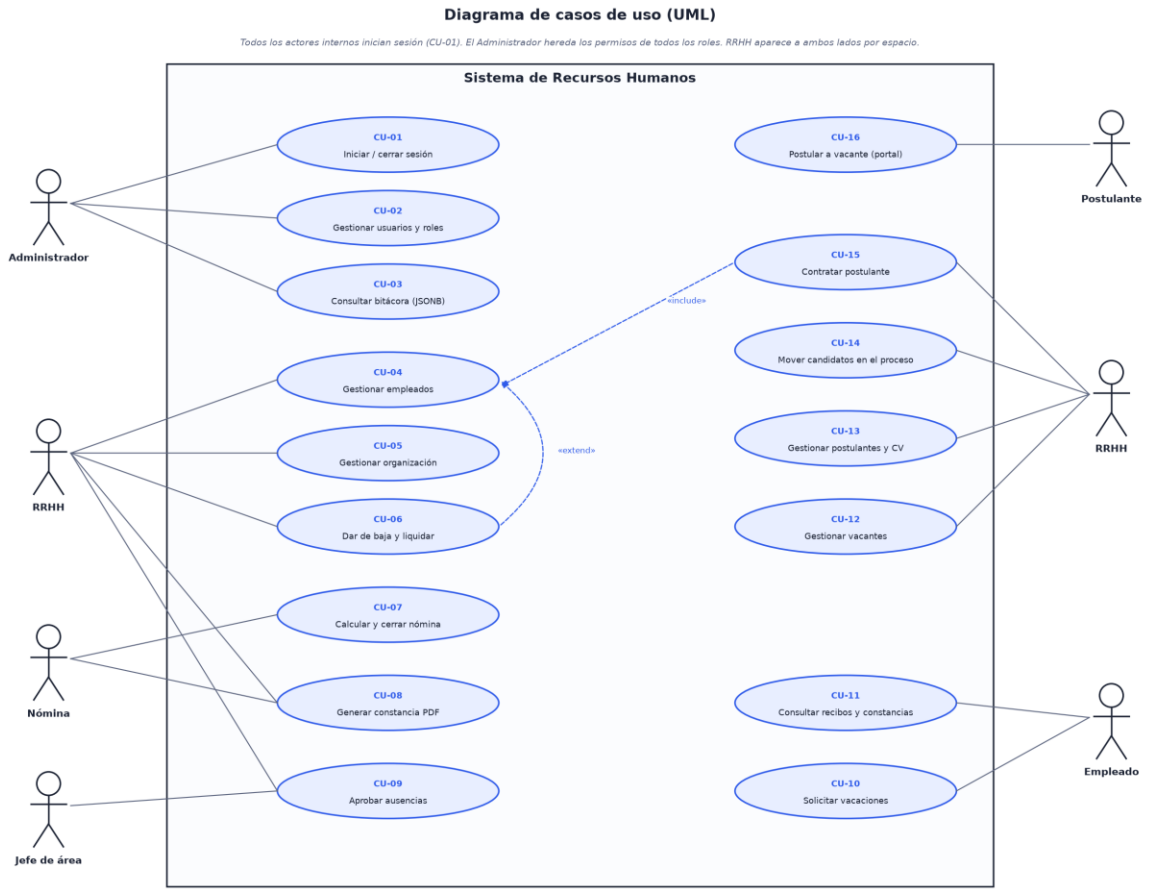


Figura 2. Diagrama global de casos de uso.

C.3 Matriz de casos de uso

ID	Caso de uso	Actor(es)	Módulo	Prioridad
CU-01	Iniciar / cerrar sesión	Todos los usuarios internos	Autenticación	Alta
CU-02	Gestionar usuarios y roles	Administrador	Administración	Alta
CU-03	Consultar bitácora de auditoría	Administrador	Administración	Media
CU-04	Gestionar empleados	RRHH	Empleados	Alta
CU-05	Gestionar organización	RRHH	Organización	Media
CU-06	Dar de baja y liquidar	RRHH	Liquidación	Media
CU-07	Calcular y cerrar nómina	Nómina	Nómina	Alta
CU-08	Generar constancia en PDF	RRHH, Nómina, Empleado	Constancias	Media
CU-09	Aprobar ausencias	Jefe de área, RRHH	Vacaciones	Media
CU-10	Solicitar vacaciones o permiso	Empleado	Vacaciones	Media
CU-11	Consultar recibos y constancias propias	Empleado	Portal del empleado	Media
CU-12	Gestionar vacantes	RRHH	Reclutamiento	Alta
CU-13	Gestionar postulantes y CV	RRHH	Reclutamiento	Alta
CU-14	Mover candidatos en el proceso	RRHH, Jefe de área (consulta)	Reclutamiento	Media
CU-15	Contratar postulante	RRHH	Reclutamiento	Alta
CU-16	Postular a una vacante	Postulante (público)	Portal de empleo	Alta

C.4 Especificación de los casos de uso principales

CU-01 — Iniciar sesión

Actores	Usuario interno (Administrador, RRHH, Nómina, Jefe, Empleado)
Descripción	El usuario se autentica con correo y contraseña para acceder a las funciones de su rol.
Precondiciones	El usuario fue invitado por un administrador, creó su contraseña y su perfil está activo.
Flujo principal	1. El usuario abre `/login`, escribe su correo y contraseña y decide si marca «Mantener sesión iniciada». 2. La SPA envía `POST /api/v1/auth/login` con `fetch`. 3. La API valida el formato de los datos y autentica contra Supabase Auth. 4. La API consulta el perfil (rol y estado) con el token del usuario. 5. La API responde 200 con el perfil y crea las cookies `rrhh_at` y `rrhh_rt` (HttpOnly, Secure, SameSite=Strict).

	6. La SPA navega con `pushState` a la página de inicio del rol (`/` o `/mi-portal`).
Flujos alternos	• **3a.** Credenciales incorrectas: la API responde 401 y la SPA muestra «Correo o contraseña incorrectos». • **4a.** Usuario desactivado: la API responde 403 y no crea cookies. • **1a.** Olvidó la contraseña: solicita un enlace (`POST /auth/recuperar`) y crea una nueva desde el correo.
Postcondiciones	Existe una sesión válida en cookies HttpOnly; el menú muestra solo las opciones permitidas para el rol.

CU-04 — Registrar un empleado (Gestionar empleados)

Actores	Recursos Humanos, Administrador
Descripción	Se crea el expediente de un nuevo empleado con su contrato inicial.
Precondiciones	Sesión activa con rol rrhh o admin. Existen departamentos y puestos.
Flujo principal	1. RRHH abre `/empleados` y pulsa «Nuevo empleado». 2. Completa datos personales, DPI, departamento, puesto, jefe, fecha de ingreso y salario. 3. La SPA valida los campos obligatorios y envía `POST /api/v1/empleados`. 4. La API sanea las entradas y ejecuta la inserción con el token del usuario; RLS verifica el rol. 5. La base de datos asigna el código (EMP-00001…) y registra la operación en la bitácora (JSONB). 6. La SPA registra el contrato inicial (`POST /api/v1/contratos`) y abre el expediente `/empleados/{id}`.
Flujos alternos	• **3a.** DPI con formato inválido (distinto de 13 dígitos): la API responde 400. • **4a.** DPI duplicado: la API responde 409 «Ya existe un registro con ese DPI». • **4b.** Usuario sin permiso: RLS rechaza la operación y la API responde 403.
Postcondiciones	El empleado y su contrato quedan registrados; la bitácora guarda el estado del registro en la columna `despues` (JSONB).

CU-07 — Calcular y cerrar la nómina

Actores	Nómina, Administrador
Descripción	Se genera la planilla de un período con los cálculos legales de Guatemala y se cierra para publicar los recibos.

Precondiciones	Sesión con rol nomina o admin; hay empleados activos y parámetros de nómina vigentes.
Flujo principal	1. El usuario crea un período (mensual, Aguinaldo o Bono 14). 2. Pulsa «Calcular nómina»: el sistema obtiene empleados, parámetros y ausencias no pagadas del período. 3. Calcula por empleado: salario devengado, horas extra (150 %), bonificación incentivo, IGSS laboral (4.83 %), retención de ISR y cuotas patronales. 4. Guarda los recibos y marca el período como «calculada». 5. El usuario ajusta horas extra, comisiones o descuentos si corresponde, y revisa totales. 6. Pulsa «Cerrar período»: el estado cambia a «cerrada».
Flujos alternos	• **2a.** No hay empleados activos en el período: se informa y no se generan recibos. • **6a.** Intento de modificar un período cerrado: un trigger de la base de datos lo impide (error 400).
Postcondiciones	Los recibos quedan inmutables y cada empleado puede verlos en su portal (RLS los muestra solo si el período está cerrado).

CU-10 — Solicitar vacaciones (con aprobación, CU-09)

Actores	Empleado (solicita), Jefe de área o RRHH (aprueba)
Descripción	El empleado solicita días de vacaciones o permiso y su jefe los aprueba o rechaza.
Precondiciones	El usuario tiene un expediente de empleado vinculado.
Flujo principal	1. El empleado abre «Mi portal → Vacaciones y permisos» y pulsa «Nueva solicitud». 2. Indica tipo, fechas, días y motivo; la SPA envía `POST /api/v1/ausencias`. 3. RLS solo permite insertar solicitudes propias y con estado «pendiente». 4. El jefe ve la solicitud en «Vacaciones y permisos» (RLS le muestra solo a su equipo). 5. El jefe pulsa «Aprobar» o «Rechazar» (`PUT /api/v1/ausencias/{id}`).
Flujos alternos	• **3a.** Un empleado intenta registrar una solicitud ya aprobada o de otra persona: RLS la rechaza (403). • **5a.** Un jefe intenta aprobar a alguien fuera de su equipo: RLS lo impide.
Postcondiciones	La solicitud queda aprobada o rechazada; las ausencias sin goce aprobadas descuentan días en la nómina.

CU-15 — Contratar un postulante

Actores	Recursos Humanos, Administrador
Descripción	Un candidato en etapa «Oferta» se convierte en empleado, copiando sus datos al esquema empresa.
Precondiciones	Existe una postulación en etapa «Oferta»; sesión con rol rrhh o admin.
Flujo principal	1. RRHH abre «Proceso de selección», elige la vacante y pulsa «Contratar» en la tarjeta del candidato. 2. Confirma departamento, puesto, salario y fecha de ingreso. 3. La SPA envía `POST /api/v1/postulaciones/{id}/contratar`. 4. La función `reclutamiento.contratar` verifica el rol, crea el empleado y su contrato (incluye CU-04) y marca la postulación como «contratado». 5. La API responde 201 con el id del nuevo empleado y la SPA abre su expediente.
Flujos alternos	• **4a.** DPI ya registrado como empleado: 409. • **4b.** Usuario sin rol autorizado: 403 «No autorizado».
Postcondiciones	Existe el empleado con `postulante_id` enlazado a su registro de reclutamiento.

CU-16 — Postular a una vacante (portal público)

Actores	Postulante (sin cuenta)
Descripción	Una persona externa envía su postulación con CV desde el portal de empleo.
Precondiciones	La vacante está abierta y marcada como publicada.
Flujo principal	1. El postulante abre `/empleos` (la SPA consulta `GET /api/v1/publico/vacantes`). 2. Pulsa «Postularme» y completa el formulario de 3 pasos; el avance se guarda en sessionStorage. 3. En el último paso adjunta su CV en PDF y acepta el tratamiento de datos personales. 4. La SPA envía `POST /api/v1/publico/postulaciones` (multipart). 5. La API valida los datos, comprueba que el archivo sea un PDF real, aplica el límite de envíos por IP y guarda postulante, CV y postulación. 6. Se muestra la confirmación y se borra el borrador de sessionStorage.
Flujos alternos	• **2a.** Recarga la página: el formulario retoma el paso y los datos guardados. • **5a.** Archivo que no es PDF, falta de consentimiento o DPI inválido: 400. • **5b.** Más de 5 envíos por hora desde la misma conexión: 429. • **5c.** Ya había postulado a la misma vacante: se informa sin duplicar el registro.

Postcondiciones	La postulación aparece en la etapa «Aplicó» del proceso de selección con su CV.

D. Diseño de Base de Datos Híbrida (PostgreSQL en Supabase)

D.1 Visión general

- **24 tablas** en tres esquemas: `empresa` (14), `reclutamiento` (9) y `public` (1, perfiles de usuario). Supera el mínimo de 10 entidades.
- **31 llaves foráneas** con reglas de borrado explícitas (CASCADE, SET NULL o RESTRICT según el caso).
- **Columnas JSONB** para documentos dinámicos (`liquidaciones.ingresos`, `liquidaciones.descuentos`) y registros de auditoría (`auditoria.antes`, `auditoria.despues`).
- **57 políticas RLS** activas (55 en las 24 tablas y 2 en Storage); todas las tablas tienen RLS habilitado y al menos una política.
- **Integridad:** restricciones CHECK (estados, tipos, DPI de 13 dígitos, calificaciones 1–5, fechas coherentes), UNIQUE (DPI, código de empleado, empleado por período) y triggers (bitácora, `updated_at`, bloqueo de períodos cerrados).

D.2 Diagrama entidad-relación — esquema «empresa»

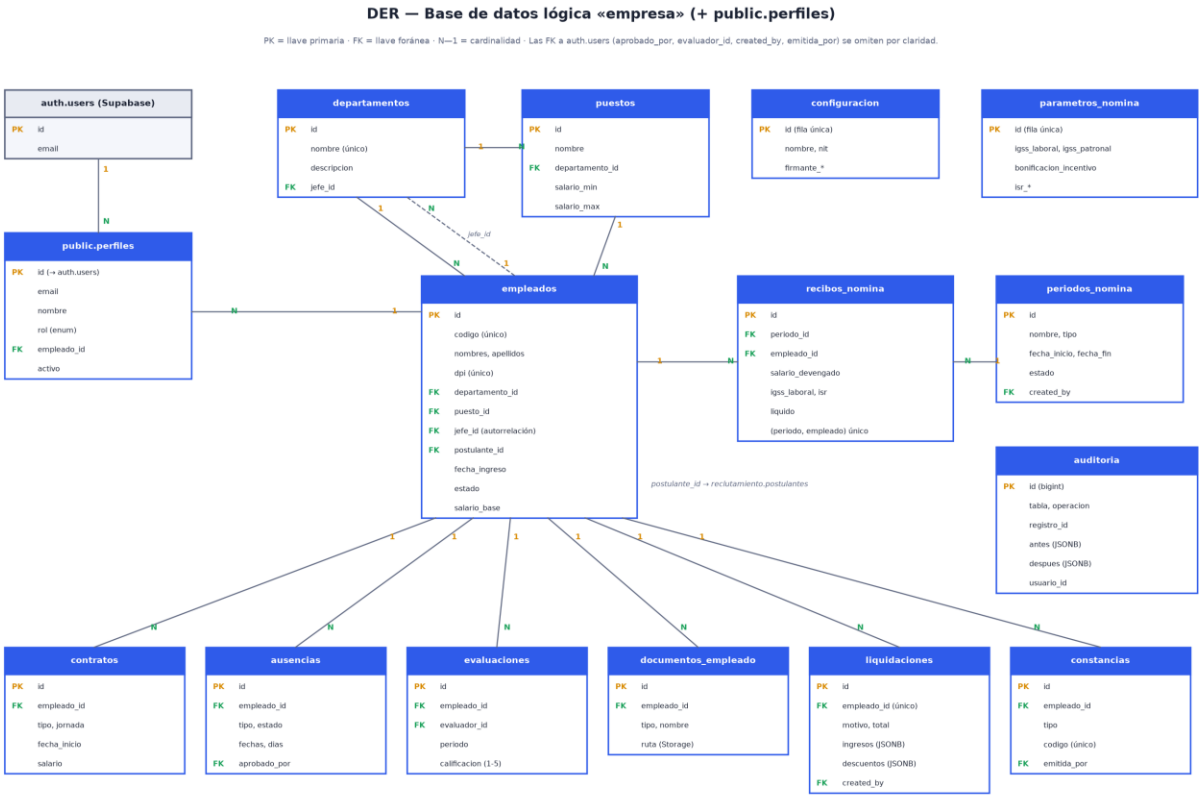


Figura 3. DER del esquema empresa (más public.perfiles).

D.3 Diagrama entidad-relación — esquema «reclutamiento»

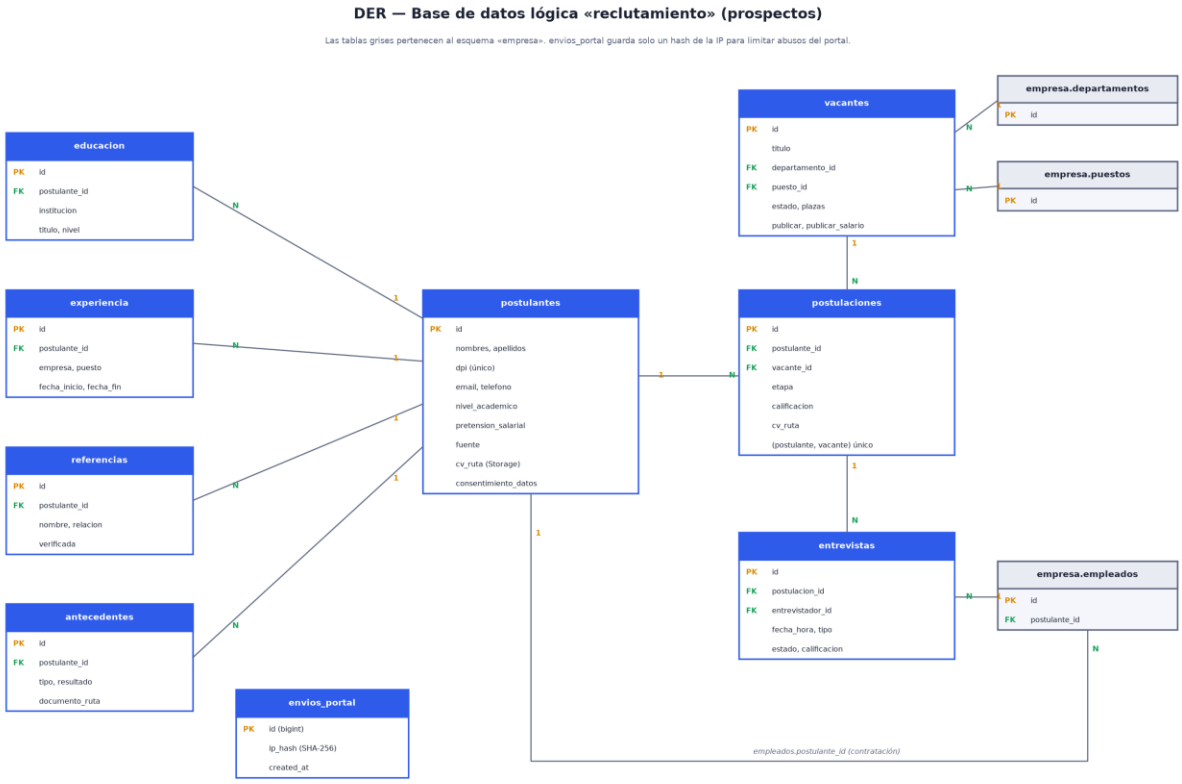


Figura 4. DER del esquema reclutamiento y sus vínculos con empresa.

D.4 Diccionario de entidades

Esquema	Tabla	Descripción	PK	Llaves foráneas
empresa	departamentos	Áreas de la empresa y su jefe	id	jefe_id → empleados
empresa	puestos	Catálogo de puestos con banda salarial	id	departamento_id → departamentos
empresa	empleados	Expediente del empleado	id	departamento_id, puesto_id, jefe_id (autorrelación), postulante_id → reclutamiento.postulantes
empresa	contratos	Historial de contratos y salarios	id	empleado_id → empleados
empresa	ausencias	Vacaciones, permisos y suspensiones	id	empleado_id → empleados; aprobado_por → auth.users
empresa	evaluaciones	Evaluaciones de desempeño	id	empleado_id → empleados; evaluador_id → auth.users

Esquema	Tabla	Descripción	PK	Llaves foráneas
empresa	documentos_empleado	Expediente digital (archivos en Storage)	id	empleado_id → empleados
empresa	parametros_nomina	Tasas legales de nómina (fila única)	id	—
empresa	periodos_nomina	Períodos de planilla	id	created_by → auth.users
empresa	recibos_nomina	Recibo por empleado y período	id	periodo_id → periodos_nomina; empleado_id → empleados
empresa	liquidaciones	Liquidación laboral (detalle en JSONB)	id	empleado_id → empleados; created_by → auth.users
empresa	configuracion	Datos de la empresa (fila única)	id	—
empresa	constancias	Constancias emitidas	id	empleado_id → empleados; emitida_por → auth.users
empresa	auditoria	Bitácora de cambios (JSONB)	id	—
reclutamiento	vacantes	Plazas abiertas	id	departamento_id, puesto_id → empresa
reclutamiento	postulantes	Candidatos y su CV	id	—
reclutamiento	educacion	Formación académica	id	postulante_id → postulantes
reclutamiento	experiencia	Experiencia laboral	id	postulante_id → postulantes
reclutamiento	referencias	Referencias personales y laborales	id	postulante_id → postulantes
reclutamiento	antecedentes	Constancias de antecedentes	id	postulante_id → postulantes
reclutamiento	postulaciones	Candidato en una vacante y su etapa	id	postulante_id → postulantes; vacante_id → vacantes
reclutamiento	entrevistas	Entrevistas programadas y resultados	id	postulacion_id → postulaciones; entrevistador_id → empresa.empleados
reclutamiento	envios_portal	Control de abuso del portal (hash de IP)	id	—
public	perfiles	Rol y estado de cada usuario	id (→ auth.users)	empleado_id → empresa.empleados

D.5 Normalización (3FN)

- **1FN:** todos los atributos son atómicos. Los datos repetitivos del postulante (estudios, experiencia, referencias, antecedentes) están en tablas propias en lugar de listas dentro de una columna.

- **2FN:** todas las tablas usan una llave primaria simple (UUID o bigint), por lo que no existen dependencias parciales.
- **3FN:** no hay dependencias transitivas. Por ejemplo, el nombre del departamento o del puesto no se copia en `empleados`: se obtiene por `departamento_id` y `puesto_id`. Las tasas legales viven en `parametros_nomina` y no se repiten.
- **Excepciones justificadas (desnormalización controlada):** `recibos_nomina.salario_base` y `liquidaciones.salario_base` guardan el salario vigente en el momento del cálculo, porque un documento de pago debe conservar sus valores históricos aunque el salario del empleado cambie después.

D.6 Esquema NoSQL / JSONB

Se usan columnas JSONB donde la estructura de los datos es variable o documental, sin necesidad de crear tablas adicionales:

Tabla.columna	Contenido	Motivo del uso de JSONB
auditoria.antes / auditoria.despues	Copia completa del registro antes y después del cambio (`to_jsonb(OLD/NEW)`)	Una sola bitácora sirve para cualquier tabla, sin importar sus columnas
liquidaciones.ingresos	Lista de conceptos: `[{ concepto, detalle, monto }]`	Cada liquidación incluye conceptos distintos según el motivo de la baja
liquidaciones.descuentos	Lista de conceptos: `[{ concepto, detalle, monto }]`	Descuentos variables (IGSS, anticipos, préstamos)

Formato de un registro de auditoría

```
{
  "id": 128,  "esquema": "empresa",  "tabla": "empleados",  "operacion": "UPDATE",
  "registro_id": "6daba4fb-3bf8-4c87-a771-c27bd91067e9",
  "antes":   { "salario_base": 6000.00, "puesto_id": "…", "estado": "activo", … },
  "despues": { "salario_base": 7500.00, "puesto_id": "…", "estado": "activo", … },
  "usuario_id": "…",  "fecha": "2026-09-24T15:02:11Z"
}
```

Formato del detalle de una liquidación

```
"ingresos": [
  { "concepto": "Indemnización", "detalle": "6 años × 8000", "monto": 48021.92 },
  { "concepto": "Aguinaldo proporcional", "detalle": "212 días", "monto": 3484.93 },
  { "concepto": "Vacaciones no gozadas", "detalle": "10 días", "monto": 2000.00 }
],
"descuentos": [ { "concepto": "IGSS laboral", "detalle": "Sobre salario pendiente", "monto":
289.80 } ]
```

Consultas JSONB que ejecuta el backend (SQL directo)

```
-- Búsqueda de texto dentro del documento JSONB de la bitácora
select * from empresa.auditoria
 where coalesce(despues, antes)::text ilike '%' || $1 || '%';

-- Campos que cambiaron entre "antes" y "despues"
select array_agg(k) from jsonb_object_keys(despues || antes) as k
 where antes -> k is distinct from despues -> k;
```

D.7 Políticas de seguridad (Row Level Security)

Las políticas usan tres funciones auxiliares `SECURITY DEFINER` con `search_path` fijo:

- `public.tiene_rol(roles…)`: indica si el usuario autenticado (`auth.uid()`) tiene alguno de los roles y está activo.
- `public.empleado_actual()`: devuelve el id del expediente vinculado al usuario.
- `empresa.es_de_mi_equipo(empleado)` y `reclutamiento.vacante_de_mi_depto(vacante)`: delimitan lo que ve un jefe.

Tabla	Política	Operación	Regla
public.perfiles	perfiles_ver	SELECT	El propio usuario, admin o rrhh
public.perfiles	perfiles_admin	UPDATE	Solo admin (cambia roles y estado)
empresa.departamentos, puestos	catalogo_ver	SELECT	Cualquier usuario autenticado
empresa.departamentos, puestos	rrhh_total	ALL	admin y rrhh
empresa.empleados	rrhh_total	ALL	admin y rrhh
empresa.empleados	nomina_ver	SELECT	Rol nomina
empresa.empleados	jefe_ver	SELECT	Jefe: solo empleados de su equipo (`es_de_mi_equipo`)
empresa.empleados	propio_ver	SELECT	El empleado ve su propio expediente
empresa.contratos	rrhh_total / nomina_ver / propio_ver	ALL / SELECT / SELECT	admin y rrhh gestionan; nomina y el propio empleado consultan
empresa.ausencias	rrhh_total	ALL	admin y rrhh
empresa.ausencias	propio_ver / propio_solicitar	SELECT / INSERT	El empleado ve las suyas y solo crea solicitudes propias en estado «pendiente»
empresa.ausencias	jefe_ver / jefe_aprobar	SELECT / UPDATE	Jefe: consulta y aprueba solo a su equipo
empresa.ausencias	nomina_ver	SELECT	Rol nomina (descuentos de planilla)
empresa.evaluaciones	rrhh_total / jefe_gestionar / propio_ver	ALL / ALL / SELECT	RRHH total; el jefe evalúa a su equipo; el empleado ve las suyas

Tabla	Política	Operación	Regla
empresa.documentos_empleado	rrhh_total / propio_ver	ALL / SELECT	RRHH gestiona; el empleado ve sus documentos
empresa.parametros_nomina	nomina_total / rrhh_lectura / todos_ver	ALL / SELECT / SELECT	admin y nomina editan; el resto solo lee (tasas legales)
empresa.periodos_nomina	nomina_total / rrhh_lectura / propio_ver	ALL / SELECT / SELECT	admin y nomina gestionan; empleados ven solo períodos cerrados
empresa.recibos_nomina	nomina_total / rrhh_lectura	ALL / SELECT	admin y nomina gestionan; rrhh consulta
empresa.recibos_nomina	propio_ver	SELECT	El empleado ve sus recibos solo si el período está cerrado
empresa.liquidaciones	gestion_total / propio_ver	ALL / SELECT	admin, rrhh y nomina gestionan; el empleado ve la suya
empresa.configuracion	todos_ver / rrhh_editar	SELECT / UPDATE	Todos leen; admin y rrhh editan
empresa.constancias	gestion_total / propio_ver / propio_solicitar	ALL / SELECT / INSERT	Gestión para admin, rrhh y nomina; el empleado emite y ve las suyas
empresa.auditoria	auditoria_admin	SELECT	Solo admin; nadie puede modificarla desde la API
reclutamiento.* (8 tablas)	rrhh_total	ALL	admin y rrhh en vacantes, postulantes, educación, experiencia, referencias, antecedentes, postulaciones y entrevistas
reclutamiento.vacantes, postulaciones, postulantes	jefe_ver	SELECT	Jefe: solo procesos de vacantes de su departamento
reclutamiento.entrevistas	jefe_entrevistas / jefe_calificar	SELECT / UPDATE	El entrevistador ve y califica sus entrevistas
reclutamiento.envios_portal	nadie_acceso (restrictiva)	ALL	Niega todo a anon y authenticated; solo el backend (service_role) accede
storage.objects	rrhh_archivos	ALL	admin y rrhh en los buckets cvs y documentos
storage.objects	propio_documentos	SELECT	El empleado descarga solo archivos de su carpeta

Prueba de las políticas: se validaron con usuarios de distintos roles (el empleado solo ve su expediente, el jefe solo ve a su equipo, un empleado no puede autoaprobar vacaciones ni cambiar su rol, y un período cerrado no puede modificarse). En la Fase 2 se demostrarán desde la consola de Supabase.

E. Especificación de la API REST (Contrato de Interfaz)

E.1 Convenciones

- **URL base:** `https://proyectodesw.netlify.app/api/v1` (mismo dominio que la SPA).
- **Formato:** JSON en UTF-8 (`Content-Type: application/json`), salvo carga de archivos (`multipart/form-data`).
- **Autenticación (navegador):** cookie HttpOnly `rrhh_at` con el JWT; el navegador la envía automáticamente y JavaScript nunca manipula el token.
- **Autenticación (otros clientes):** la API también acepta `Authorization: Bearer <JWT>` (por ejemplo, desde Postman o curl). Como esa cabecera no la adjunta el navegador por sí solo, en ese caso no se exige el token CSRF.
- **Cabeceras anti-CSRF:** toda petición que modifica datos (POST, PUT, DELETE) debe incluir `X-Requested-With: fetch`; si usa la cookie de sesión, además `X-CSRF-Token` con el valor que entregan `/auth/login` y `/auth/sesion`.
- **Cabecera Content-Type:** `application/json` en los cuerpos JSON; si falta o es otra, la API responde 400.
- **Recursos:** `departamentos, puestos, empleados, contratos, ausencias, evaluaciones, documentos_empleado, parametros_nomina, periodos_nomina, recibos_nomina, liquidaciones, configuracion, constancias, auditoria, vacantes, postulantes, educacion, experiencia, referencias, antecedentes, postulaciones, entrevistas, perfiles` (lista blanca; cada uno con sus métodos permitidos).
- **Filtros:** parámetro `filtros` con un arreglo JSON de `[columna, operador, valor]`. Operadores permitidos: eq, neq, gt, gte, lt, lte, in, ilike, is, or.

E.2 Códigos de respuesta

Código	Significado en la API
200 OK	Consulta, actualización o eliminación exitosa
201 Created	Registro, archivo o postulación creados
400 Bad Request	Datos inválidos: formato, campo inexistente, filtro no permitido, regla de negocio
401 Unauthorized	Sin sesión o sesión vencida
403 Forbidden	Sin permiso (rol o RLS), o petición rechazada por la protección CSRF
404 Not Found	Recurso, ruta o registro inexistente (o no visible para el usuario)
405 Method Not Allowed	Método no permitido para ese recurso
409 Conflict	Duplicado (DPI, correo) o registro con dependencias
413 Payload Too Large	Archivo mayor a 5 MB
429 Too Many Requests	Límite de postulaciones por hora superado
500 Internal Server Error	Error inesperado (el detalle solo se registra en el servidor)

E.3 Matriz de endpoints

Verbo	Ruta	Cabeceras	Payload / parámetros	Respuestas
POST	/api/v1/auth/login	Content-Type, X-Requested-With	{ email, password, recordar }	200 perfil + csrf + cookies · 400 · 401 · 403
GET	/api/v1/auth/sesion	Cookie	—	200 { usuario, perfil, csrf } o { usuario: null }
POST	/api/v1/auth/logout	Cookie, X-Requested-With	{ todos }	200 (borra cookies)
POST	/api/v1/auth/recuperar	Content-Type, X-Requested-With	{ email }	200 · 400
POST	/api/v1/auth/verificar	Content-Type, X-Requested-With	{ token_hash, type }	200 + cookies · 400
PUT	/api/v1/auth/password	Cookie, Content-Type, X-Requested-With, X-CSRF-Token	{ password }	200 · 400 · 401
GET	/api/v1/{recurso}	Cookie	Query: select, filtros, orden, limite, conteo	200 { datos } · 400 · 401 · 403
GET	/api/v1/{recurso}/{id}	Cookie	Query: select	200 { datos } · 401 · 404
POST	/api/v1/{recurso}	Cookie, Content-Type, X-Requested-With, X-CSRF-Token	{ datos, select?, conflicto? }	201 { datos } · 400 · 401 · 403 · 409
PUT	/api/v1/{recurso}/{id}	Cookie, Content-Type, X-Requested-With, X-CSRF-Token	{ datos }	200 { datos } · 400 · 401 · 403
DELETE	/api/v1/{recurso}/{id}	Cookie, X-Requested-With, X-CSRF-Token	—	200 { ok } · 401 · 403 · 409
POST	/api/v1/usuarios	Cookie, Content-Type, X-Requested-With, X-CSRF-Token	{ email, nombre, rol, empleado_id }	201 · 400 · 403 · 409
DELETE	/api/v1/usuarios/{id}	Cookie, X-Requested-With, X-CSRF-Token	—	200 · 400 · 403 · 404
POST	/api/v1/postulaciones/{id}/contratar	Cookie, Content-Type, X-	{ departamento_id, puesto_id, salario, fecha_ingreso }	201 { empleado_id } · 403 · 409

Verbo	Ruta	Cabeceras	Payload / parámetros	Respuestas
		Requested-With, X-CSRF-Token		
POST	/api/v1/archivos/{bucket}	Cookie, X-Requested-With, X-CSRF-Token	multipart: carpeta, archivo	201 { ruta } · 400 · 413
GET	/api/v1/archivos/{bucket}/url	Cookie	Query: ruta	200 { url } · 404
DELETE	/api/v1/archivos/{bucket}	Cookie, X-Requested-With, X-CSRF-Token	Query: ruta o carpeta	200 · 400 · 403
GET	/api/v1/reportes/resumen	Cookie	—	200 { datos } · 401
GET	/api/v1/reportes/bitacora	Cookie	Query: buscar, tabla, operacion, limite	200 { datos } · 400 · 401
GET	/api/v1/publico/vacantes	—	—	200 { datos }
GET	/api/v1/publico/empresa	—	—	200 { datos }
POST	/api/v1/publico/postulaciones	X-Requested-With	multipart: datos, cv (PDF), consentimiento	201 · 400 · 404 · 413 · 429

E.4 Ejemplos

Inicio de sesión

```
POST /api/v1/auth/login
Content-Type: application/json
X-Requested-With: fetch

{ "email": "rrhh@empresa.com", "password": "********", "recordar": false }

HTTP/1.1 200 OK
Set-Cookie: rrhh_at=eyJ…; Path=/api; HttpOnly; Secure; SameSite=Strict; Max-Age=3600
Set-Cookie: rrhh_rt=…; Path=/api; HttpOnly; Secure; SameSite=Strict
{ "usuario": { "id": "…", "email": "rrhh@empresa.com" }, "perfil": { "rol": "rrhh", "activo":
true }, "csrf": "Q3J…" }
```

Consulta con filtros

```
GET
/api/v1/empleados?select=id,nombres,apellidos&filtros=[["estado","eq","activo"]]&orden=apellido
s.asc

HTTP/1.1 200 OK
{ "datos": [ { "id": "…", "nombres": "Ana", "apellidos": "López" } ] }
```

Modificación con token CSRF

```
PUT /api/v1/empleados/6daba4fb-…
Cookie: rrhh_at=eyJ…
Content-Type: application/json
X-Requested-With: fetch
X-CSRF-Token: Q3J…

{ "datos": { "telefono": "5555-1234" } }

HTTP/1.1 200 OK
{ "datos": [ { "id": "6daba4fb-…", "telefono": "5555-1234" } ] }
```

Consulta desde Postman con Authorization

```
GET /api/v1/departamentos
Authorization: Bearer eyJ…

HTTP/1.1 200 OK
{ "datos": [ { "id": "…", "nombre": "Recursos Humanos" } ] }
```

Error estructurado

```
HTTP/1.1 409 Conflict
{ "error": { "codigo": "DUPLICADO", "mensaje": "Ya existe un registro con ese DPI." } }
```

F. Plan de Seguridad y Mitigaciones OWASP

F.1 Manejo de sesiones y autenticación

1. El usuario envía sus credenciales a la API propia; la SPA nunca se comunica con Supabase directamente.
2. La API autentica con **Supabase Auth**, que emite un JWT de acceso (1 hora) y un token de renovación.
3. La API guarda ambos tokens en **cookies HttpOnly, Secure y SameSite=Strict** con `Path=/api`. La respuesta JSON nunca incluye los tokens.
4. En cada petición, la API lee la cookie, verifica la vigencia del JWT y, si venció, lo **renueva automáticamente** con el token de renovación.
5. Las consultas a Supabase se hacen **con el JWT del usuario**, de modo que las políticas RLS se aplican siempre.
6. Junto con la sesión, la API entrega un **token anti-CSRF**: un HMAC-SHA256 del identificador de la sesión firmado con un secreto del servidor. La SPA lo guarda solo en memoria y lo envía en `X-CSRF-Token` en cada modificación; la API lo compara en tiempo constante.
7. Al cerrar sesión se revoca el token de renovación (local o en todos los dispositivos) y se borran las cookies.

Atributo de la cookie	Protección que aporta
HttpOnly	JavaScript no puede leer el token: un ataque XSS no puede robar la sesión
Secure	La cookie solo viaja por HTTPS
SameSite=Strict	El navegador no la envía en peticiones iniciadas desde otros sitios (mitiga CSRF)
Path=/api	Solo se envía a la API, no a los archivos estáticos
Max-Age	Acceso: 1 hora. Renovación: sesión del navegador, o 30 días con «Mantener sesión iniciada»

F.2 Matriz de mitigación de vulnerabilidades

Amenaza (OWASP Top 10)	Mitigación en el cliente	Mitigación en backend / base de datos
XSS — Cross-Site Scripting (A03)	Todo dato dinámico se inserta escapado (`esc()`); sin `innerHTML` con datos sin escapar; Content-Security-Policy `script-src 'self'` sin scripts en línea	Sanitización de entradas: se eliminan etiquetas HTML y caracteres de control; validación de tipo y longitud; cookies HttpOnly
CSRF — Cross-Site Request Forgery	Las peticiones de cambio envían `X-Requested-With: fetch` y el token `X-CSRF-Token`, guardado solo en memoria	Validación del token CSRF (HMAC ligado a la sesión); cookies SameSite=Strict; la API rechaza (403) peticiones sin cabecera, sin token válido o con un Origin distinto; sin CORS habilitado

Amenaza (OWASP Top 10)	Mitigación en el cliente	Mitigación en backend / base de datos
Inyección SQL (A03)	La SPA no arma SQL	Lista blanca de recursos, columnas validadas con expresión regular, operadores permitidos; consultas parametrizadas (PostgREST y SQL con parámetros $1)
Control de acceso roto (A01)	El menú y el router ocultan lo que el rol no puede usar	RBAC con 5 roles y 57 políticas RLS en la base de datos: la regla se cumple aunque se manipule el cliente
Fallas criptográficas / secretos (A02)	El navegador no contiene claves de Supabase	SUPABASE_SERVICE_ROLE_KEY y DATABASE_URL solo como variables secretas del servidor; .env excluido de Git; HTTPS en todo el sitio
Diseño inseguro de carga de archivos	Validación de tipo y tamaño antes de enviar	Se verifica el contenido real (firma PDF/JPG/PNG), no solo la extensión; límite de 5 MB; buckets privados con URL firmadas de 1 hora
Abuso del portal público / bots	Campo trampa oculto; tiempo mínimo de llenado	Límite de 5 postulaciones por hora por IP (se guarda solo un hash SHA-256); validación completa en el servidor
Identificación y autenticación (A07)	Contraseñas de 8+ caracteres con mayúscula, minúscula y número	Registro público desactivado (solo por invitación); política de contraseñas en Supabase Auth; revocación de sesiones
Configuración de seguridad (A05)	—	Cabeceras X-Content-Type-Options, X-Frame-Options (DENY), Referrer-Policy y CSP; funciones SQL sin EXECUTE para usuarios anónimos
Registro y monitoreo (A09)	—	Bitácora de auditoría con triggers (antes/después en JSONB) para empleados, contratos, nómina, postulantes, antecedentes, perfiles y eliminación de usuarios

F.3 Manejo seguro de secretos

Secreto	Dónde vive	Quién lo usa
Clave pública (anon/publishable)	Variable de entorno del servidor	API: operaciones con el JWT del usuario
SUPABASE_SERVICE_ROLE_KEY	Variable secreta de Netlify	API: invitar y eliminar usuarios, portal de empleo (tras validar el rol o los datos)
DATABASE_URL (contraseña de PostgreSQL)	Variable secreta de Netlify	API: consultas SQL directas, que adoptan la identidad del usuario para respetar RLS
Contraseña de aplicación del correo (SMTP)	Configuración de Supabase Auth	Envío de invitaciones y recuperación de contraseña

Ningún secreto se guarda en el repositorio: `.gitignore` excluye `.env`, y `.env.example` documenta las variables sin valores.

Anexo. Referencias del proyecto

Elemento	Referencia
Aplicación publicada	https://proyectodesw.netlify.app
Portal de empleo	https://proyectodesw.netlify.app/empleos
Repositorio	https://github.com/Antonio-Sj04/sistema-rrhh (privado)
Migraciones SQL	supabase/migrations/*.sql
API REST	server/ (rutas, validación, cookies, SQL directo) y netlify/functions/api.mjs
Pruebas automáticas	tests/ — 24 casos (nómina, montos en letras, validación y seguridad de la API)